Пособие для изучения Python

для подготовки к ОГЭ и ЕГЭ от Алины Лисянской

Содержание

Предисловие **1**

Введение в Python **1**

Переменные 1
Типы данных 2
Команды print и input 2
Арифметические операции 3
Перевод чисел в другие СС 4

Условный оператор **5**

Циклы **6**

Цикл while 6
Цикл for 7

Предисловие

Это пособие для повторение теории в Python. Оно поможет вам систематизировать знания, повторить синтаксис и освоить приёмы работы в Python, которые ускоряют написание кода на экзамене

Введение

Языки программирования бывают интерпретируемыми и компилируемыми. Компилируемые сначала переводят всю программу целиком в машинный код, а потом запускают. Интерпретируемые выполняют программу строчка за строчкой прямо во время работы. Python — интерпретируемый. Отсюда вытекают его особенности: он медленнее многих других языков, но прост в изучении и имеет множество применений!

Переменные

Переменная – это именованная ячейка памяти, в которой хранится какое-то значение. Создать переменную просто: пишем имя, знак равно и значение:

```
x = 5
name = "Аня"
```

Имя переменной может быть любым, но оно не может начинаться с цифры и не может содержать пробелы. Мы будем стараться называть переменные так, чтобы они отражали значение, которое хранят.

Типы данных

У каждого значения есть тип. Основные типы в Python:

- `int` — целое число;
- `float` — дробное число;
- `str` — строка, то есть текст;
- `bool` — логическое значение (`True` или `False`).

Python сам определяет тип, когда мы создаём переменную:

```
a = 10          # int
b = 3.14        # float
c = "Привет"    # str
```

Иногда нам потребуется преобразовать один тип в другой. Для этого используют `int()`, `float()`, `str()`:

```
int("5")    # строку "5" -> число 5
str(42)     # число 42 -> строку "42"
```

Основные команды

`print()` выводит что-либо на экран. Можно передать текст в кавычках, число, посчитать арифметическое выражение, вывести значение переменной или сразу нескольких через запятую (в таком случае они выведутся через пробел):

```
print("Привет!")            # Привет!
print(2 + 2)                # 4
print("Сумма:", 2 + 2)      # Сумма: 4
print(a, b, c)              # 10 3.14 Привет
```

input() считывает то, что пользователь вводит с клавиатуры. Важно: `input()` всегда возвращает строку, даже если пользователь ввёл число. Поэтому если нужно, чтобы пользователь ввел не текст, а число, указываем нужный тип данных перед `input()`:

```
name   = input()    # str
age    = int(input())  # int
weight = float(input())   # float
```

Арифметические операции

Познакомимся с основными арифметическими операциями. Все кроме деления здесь тривиальны:

```
a = 10
b = 3

a + b   #сложение -> 13
a - b   #вычитание -> 7
a * b   #умножение -> 30
a ** b  #возведение в степень -> 1000
```

С делением интереснее:

```
a / b   #дробное деление -> 3.3333...
a // b  #целочисленное деление -> 3
a % b   #остаток от деления -> 1
```

Обратите внимание: обычное деление `/` всегда возвращает `float`, даже если числа делятся нацело:

```
10 / 2   #5.0 (а не 5)!
10 // 2  #5  (целочисленное деление вернёт int)
```

Операция остатка от деления `%` очень часто используется в задачах, например чтобы проверить чётность числа:

```
if n % 2 == 0:
    print("чётное")
else:
    print("нечётное")
```

Для удобства в Python есть **сокращённые операторы присваивания**:

```
x = 10
x += 5   # то же самое, что x = x + 5  -> 15
x -= 3   # то же самое, что x = x - 3  -> 12
x *= 2   # то же самое, что x = x * 2  -> 24
x //= 4  # то же самое, что x = x // 4 -> 6
x %= 4   # то же самое, что x = x % 4  -> 2
```

Порядок выполнения операций такой же, как в математике.

Также помним **про работу со строками**. Строки можно складывать.

```
s1 = 'pine'
s2 = 'apple'
print(s1 + s2)  #'pineapple'
```

А также умножать на число, по сути дублируя строку указанное количество раз:

```
t1 = 'hi'
print(t1 * 3)  #'hihihi'
print(5 * t1) #'hihihihihi'
```

Перевод чисел: bin(), oct(), hex()

Это встроенные функции для перевода чисел в другие системы счисления, которые нужны для некоторых заданий в ОГЭ и ЕГЭ.

- **bin(n)** переводит число в двоичную систему
- **oct(n)** – в восьмеричную
- **hex(n)** – в шестнадцатеричную

```
bin(10)   # '0b1010' - префикс 0b: двоичная
oct(8)    # '0o10'   - префикс 0o: восьмеричная
hex(255)  # '0xff'   - префикс 0x: шестнадцатеричная
```

Если нужно получить число без префикса, используем срез строки (про них подробнее чуть позже):

```
bin(10)[2:]   # '1010'
hex(255)[2:]  # 'ff'
```

Обратный перевод (из любой системы в десятичную) делает функция **int()** с двумя аргументами: первый – строка с числом, второй – основание системы счисления:

```
int('1010', 2)    # 10  (из двоичной)
int('ff', 16)     # 255 (из шестнадцатеричной)
int('17', 8)      # 15  (из восьмеричной)
```

Условный оператор

Условный оператор позволяет программе принимать решения: выполнять один блок кода или другой в зависимости от условия. Основная конструкция:

```
if условие:
    #команда 1 выполнится, если условие истинно
elif другое_условие:
    #команда 2 выполнится, если первое условие ложно, а это истинно
else:
    #команда 3 выполнится, если все условия выше ложны
```

Блоки кода внутри `if` обязательно выделяются отступом – это называют табуляцией, равной 4 пробела. Python определяет принадлежность строки к блоку именно по этому отступу.

Пример:

```
age = int(input("Введи возраст: "))

if age < 0:
    print("Неверный возраст")
elif age < 18:
    print("Несовершеннолетний")
else:
    print("Совершеннолетний")
```

Ветки `elif` и `else` необязательны. Имеется ввиду, что можно написать просто `if` и без них.

В условиях используются операторы сравнения:

```
a == b   # a равно b
a != b   # a не равно b
a > b    # a больше b
a < b    # a меньше b
a >= b   # a больше или равно b
a <= b   # a меньше или равно b
```

Несколько условий можно объединять логическими операторами `and` (и), `or` (или), `not` (не):

```
x = 15
if x > 10 and x < 20:
    print("x находится между 10 и 20")
```

Циклы

Цикл while

Цикл `while` выполняет блок кода до тех пор, пока условие истинно:

```
while условие:
    # тело цикла
```

Пример – считаем от 1 до 5:

```
i = 1
while i <= 5:
    print(i)
    i += 1   # увеличиваем i на 1, иначе цикл бесконечный
```

Важно следить за тем, чтобы условие когда-нибудь стало ложным – иначе программа зависнет в бесконечном цикле.

Команда `break` прерывает цикл досрочно, а `continue` пропускает текущую итерацию и переходит к следующей:

```
i = 0
while True:
    i += 1
    if i % 2 == 0:
        continue   # пропускаем чётные
    if i > 9:
        break      # останавливаемся после 9
    print(i)       # выведет 1, 3, 5, 7, 9
```

Цикл for

Цикл `for` перебирает элементы какой-либо последовательности – строки, списка, диапазона чисел. На каждой итерации переменная цикла принимает очередное значение из последовательности:

```
for переменная in последовательность:
    # тело цикла
```

Разберём на примере. Напишем `for i in range(5)` — это значит: «пусть переменная `i` по очереди принимает значения 0 до 5 не включительно, и для каждого из них выполняй тело цикла»:

```
for i in range(5):
    print(i)    # выведет 0, затем 1, затем 2, затем 3, затем 4
```

Функция `range()` генерирует диапазон чисел и принимает от одного до трёх аргументов:

```
range(stop)           # от 0 до stop - 1
range(start, stop)    # от start до stop - 1
range(start, stop, step)  # от start до stop - 1 с шагом step
```

Важно запомнить: правая граница **не включается**. То есть `range(3, 6)` даст числа 3, 4, 5.

Пример с тремя аргументами:

```
for i in range(0, 10, 2):
    print(i)    # 0, 2, 4, 6, 8

for i in range(10, 0, -1):
    print(i)    # 10, 9, 8, ..., 1 - отрицательный шаг, идём в обратном порядке
```

Отрицательный шаг особенно полезен, когда нужно идти от большего к меньшему. При этом правило то же: правая граница не включается, поэтому `range(10, 0, -1)` даст числа от 10 до 1, но не 0.

Цикл `for` тоже поддерживает `break` и `continue`.